Danmarks
Tekniske
Universitet

Execise 1
(Ethernet Frame Check Sequence)

FPGA design for communication systems(34349)

Author

Hákon Hlynsson - s225765

March 4, 2026

Contents

1 Introduction **1**

2 Serial implementation **3**

3 Synthesis and Place & route **5**

4 Testbench serial implementation **5**

5 Parallel implementation **7**

6 Synthesis and Place & route **10**

7 Testbench Parallel implmentation **10**

References **13**

8 Appendix **14**

8.1 Matlab for generation of CRC value 14

8.2 Matlab for generating the parallel setup 15

1 Introduction

When sending packages over the internet, it is important to ensure that the data the receiver receives is the exact same data you intended to transmit. But how do you verify that the data is correct without knowing the original content beforehand?

One way to verify data integrity is by using a Cyclic Redundancy Check (CRC). It works by taking your data and adding a Frame Check Sequence (FCS) to the end of the package, as illustrated below.

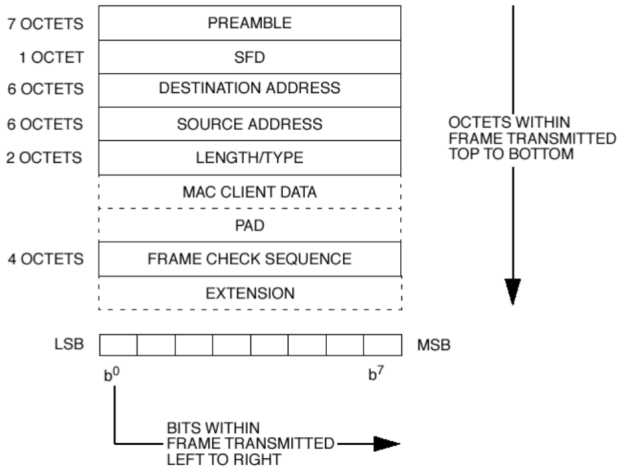

Figure 1: Shows an entire ethernet frame

This sequence is not just a collection of random bits; it is calculated directly from the data you intend to send. The process relies on a generator polynomial, $G(x)$, which is used to encode the data into the Frame Check Sequence (FCS)

$$G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^{8} + x^{7} + x^{5} + x^{4} + x^{2} + x + 1$$

The encoding process works as following

- The first 32 bits of the data are inverted.
- Add additional 32 zero-bits are appended to the end of the original data.
- Traverse the data bit by bit and whenever a '1' is encountered, the next 32 bits are XORed with the generator polynomial(if '0' then nothing happens).This continues until the last 32 bits.
- The resulting remainder in the last 32 bits is then inverted. These final 32 bits becomes the Frame Check Sequence that is attached to the end of the packet.

Once the FCS has been calculated, the complete package is ready to be sent to the receiver. To see an implementation of CRC calculater in Matlab see appendix.

Now that we understand how to prepare the data using CRC before transmission, we must consider how to check the data upon receipt. To determine if the received data is correct, we use a shift register circuit, as shown below. The circuit operates by shifting the package

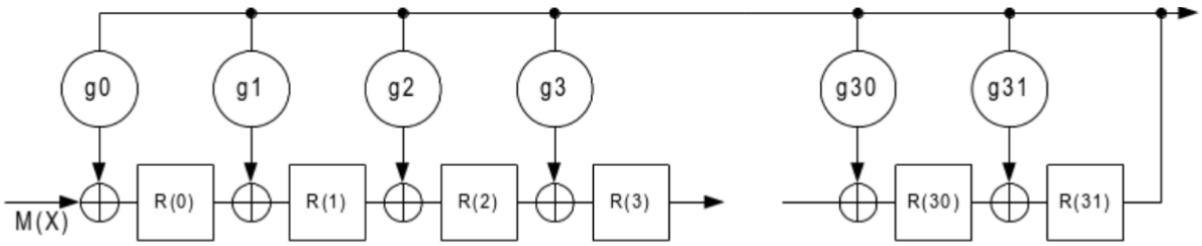

Figure 2: Shift Register to check CRC value

in serially, bit by bit, as the message $M(x)$. Each bit is XORed with the output of the last register, which has been ANDed with the corresponding coefficient of the generator polynomial $G(x)$. The next register then takes the output of previous register and XORed with the output of the last register, which has been ANDed with the corresponding coefficient of the generator polynomial $G(x)$. This continues all the way to the last register. But how is this actually used?

Just as in the encoding process, the first and last 32 bits of the packet are inverted before being inserted into the circuit. If the data received is identical to the data transmitted, the final state of all registers will be zero. However, if even a single bit is flipped during transmission, the registers will contain a value different from zero. Therefore, if the registers contain all zeros after processing, we can conclude that the intended data was received correctly.

2 Serial implementation

In the following chapter we are going to be going throghe, the serial implementation of an FCS circuit like described in introduktion. The code that has been implemented can be seen below or in github.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity fcs_check_serial is
  port (
    clk            : in std_logic;-- system clock
    reset          : in std_logic;-- asynchronous reset
    start_of_frame : in std_logic;-- arrival of the first bit.
    end_of_frame   : in std_logic;-- arrival of the first bit in FCS.
    data_in        : in std_logic;-- serial input data.
    fcs_error      : out std_logic-- indicates an error
  );
end fcs_check_serial;

architecture behavioral of fcs_check_serial is
  signal Reg            : std_logic_vector(31 downto 0);
  signal Counter        : unsigned(4 downto 0);
  signal Data_insert    : std_logic;
  signal Checksum_Start : std_logic;

begin

-- Data insert process
process(clk, start_of_frame, end_of_frame, data_in)
begin
  if(start_of_frame = '1' or end_of_frame = '1' or Counter < 31) then
    Data_insert <= not data_in;
  else
    Data_insert <= data_in;
  end if;
end process;

-- Serial FCS setup
process(clk, reset)
begin
  if (reset = '1') then
    Reg            <= (others => '0');
    Counter        <= (others => '0');
    fcs_error      <= 'X';
    Checksum_Start <= '0';
```

```
  elsif rising_edge(clk) then
    if (end_of_frame = '1') then
      Checksum_Start <= '1';
    end if;

    if (start_of_frame = '1' or end_of_frame = '1') then
      Counter <= (others => '0');
    elsif(Counter < 31) then
      Counter <= Counter + 1;
    end if;

    Reg(0)  <= Data_insert xor Reg(31);
    Reg(1)  <= Reg(0)  xor Reg(31);
    Reg(2)  <= Reg(1)  xor Reg(31);
    Reg(3)  <= Reg(2);
    Reg(4)  <= Reg(3)  xor Reg(31);
    Reg(5)  <= Reg(4)  xor Reg(31);
    Reg(6)  <= Reg(5);
    Reg(7)  <= Reg(6)  xor Reg(31);
    Reg(8)  <= Reg(7)  xor Reg(31);
    Reg(9)  <= Reg(8);
    Reg(10) <= Reg(9)  xor Reg(31);
    Reg(11) <= Reg(10) xor Reg(31);
    Reg(12) <= Reg(11) xor Reg(31);
    Reg(13) <= Reg(12);
    Reg(14) <= Reg(13);
    Reg(15) <= Reg(14);
    Reg(16) <= Reg(15) xor Reg(31);
    Reg(17) <= Reg(16);
    Reg(18) <= Reg(17);
    Reg(19) <= Reg(18);
    Reg(20) <= Reg(19);
    Reg(21) <= Reg(20);
    Reg(22) <= Reg(21) xor Reg(31);
    Reg(23) <= Reg(22) xor Reg(31);
    Reg(24) <= Reg(23);
    Reg(25) <= Reg(24);
    Reg(26) <= Reg(25) xor Reg(31);
    Reg(27) <= Reg(26);
    Reg(28) <= Reg(27);
    Reg(29) <= Reg(28);
    Reg(30) <= Reg(29);
    Reg(31) <= Reg(30);

    --checking if the calculated value is 0
    if (Checksum_Start = '1' and Counter = 31) then
```

```
      if (Reg = X"00000000") then
        fcs_error <= '0';
      else
        fcs_error <= '1';
      end if;
      Checksum_Start <= '0';
    end if;
  end if;
end process;
end behavioral;
```

Listing 1: Serial FCS Check Module

3 Synthesis and Place & route

A synthesis and place & route were also performed, which determined that the worst case operating frequency was 370.1 MHz at 1200mV and 85°C.

4 Testbench serial implementation

The testbench used for verification is shown below and can also be accessed via github.

```
library ieee;
use ieee.std_logic_1164.all;

entity fcs_check_serial_tb is
end fcs_check_serial_tb;

architecture behavior of fcs_check_serial_tb is

  -- component used in testbench
  component fcs_check_serial
    port(clk            : in  std_logic;
         reset          : in  std_logic;
         start_of_frame : in  std_logic;
         end_of_frame   : in  std_logic;
         data_in        : in  std_logic;
         fcs_error      : out std_logic);
  end component;

  -- Signals
  signal Test_clk            : std_logic := '0';
  signal Test_reset          : std_logic := '0';
  signal Test_start_of_frame : std_logic := '0';
  signal Test_end_of_frame   : std_logic := '0';
```

```
  signal Test_data_in        : std_logic := '0';
  signal Test_fcs_error      : std_logic;

  -- Clock Speed
  constant clk_period : time := 1 ns;

  -- Ethernet packet (FCS checksum E6 C5 3D B2)
  constant Ethernet_packet : std_logic_vector(511 downto 0) :=
    x"00_10_A4_7B_EA_80_00_12_34_56_78_90_08_00_45" &
    x"00_00_2E_B3_FE_00_00_80_11_05_40_C0_A8_00_2C" &
    x"C0_A8_00_04_04_00_04_00_00_1A_2D_E8_00_01_02" &
    x"03_04_05_06_07_08_09_0A_0B_0C_0D_0E_0F_10_11" &
    x"E6_C5_3D_B2";

begin

  Comp1 : fcs_check_serial port map (
    clk            => Test_clk,
    reset          => Test_reset,
    start_of_frame => Test_start_of_frame,
    end_of_frame   => Test_end_of_frame,
    data_in        => Test_data_in,
    fcs_error      => Test_fcs_error
  );

  -- Clock generation
  Clk_Generator : process
  begin
    Test_clk <= '0';
    wait for clk_period / 2;
    Test_clk <= '1';
    wait for clk_period / 2;
  end process;

  -- Data_Simulation
  Data_Sim : process
  begin
    -- Start by resetting everything
    Test_reset <= '1';
    wait for clk_period;
    Test_reset <= '0';

    -- Begin sending the data
    for i in Ethernet_packet'range loop
      if i = (Ethernet_packet'length - 1) then
        Test_start_of_frame <= '1';
```

```
      else
        Test_start_of_frame <= '0';
      end if;

      Test_data_in <= Ethernet_packet(i);

      if i = 31 then -- goes high when 32 bits remain (FCS)
        Test_end_of_frame <= '1';
      else
        Test_end_of_frame <= '0';
      end if;

      wait for clk_period;
    end loop;

    Test_data_in <= '0';
    wait;
  end process;
end behavior;
```

Listing 2: Serial FCS Check Testbench

5 Parallel implementation

The following chapter details the parallel implementation of an FCS circuit. To understand the logic assigned to each register, please refer to the calculations in the appendix. The implemented code is provided below or via github.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity fcs_check_parallel is
  port (
    clk            : in std_logic;      -- system clock
    reset          : in std_logic;      -- asynchronous reset
    start_of_frame : in std_logic;      -- arrival of the first byte.
    end_of_frame   : in std_logic;      -- arrival of the first byte
   in FCS.
    data_in      : in std_logic_vector(7 downto 0);   -- input data.
    fcs_error      : out std_logic      -- indicates an error.
  );
end fcs_check_parallel;
architecture behavioral of fcs_check_parallel is
  signal Reg           : std_logic_vector(31 downto 0); -- The 32
   registers used to store data
```

```
  signal Counter        : unsigned(1 downto 0);     -- Counter that counts number of times 8 bits have been sendt
  signal Data_insert    : std_logic_vector(7 downto 0);  -- Data that should be inserted
  signal Checksum_Start: std_logic;      -- checksum goes high when the checksum is being inserted

begin

  -- Data insert process (The first and last 32 bits has due to the way  )
  process(start_of_frame,end_of_frame,data_in,Counter)
  Begin
    if(start_of_frame = '1' or end_of_frame = '1' or Counter < 3) then
      Data_insert <= not data_in;
    else
      Data_insert <= data_in;
    end if;
  end process;

  -- Parallel FCS setup
  process(clk,reset)
  Begin
    if (reset = '1') then -- reset the registor and the
      Reg      <= (others => '0');
            Counter <= (others => '0');
      fcs_error    <= '1';

    elsif rising_edge(clk) then
      if (end_of_frame = '1') then
        Checksum_Start <= '1';
      end if;

      if (start_of_frame = '1' or end_of_frame = '1') then
        Counter <= (others => '0');
      elsif(Counter < 3) then
        Counter <= Counter + 1;
      end if;

      Reg(0) <= Reg(24) xor Reg(30) xor Data_insert(0);
      Reg(1) <= Reg(24) xor Reg(25) xor Reg(30) xor Reg(31) xor Data_insert(1);
      Reg(2) <= Reg(24) xor Reg(25) xor Reg(26) xor Reg(30) xor Reg(31) xor Data_insert(2);
      Reg(3) <= Reg(25) xor Reg(26) xor Reg(27) xor Reg(31) xor
```

```
Data_insert(3);
   Reg(4) <= Reg(24) xor Reg(26) xor Reg(27) xor Reg(28) xor Reg(30) xor Data_insert(4);
   Reg(5) <= Reg(24) xor Reg(25) xor Reg(27) xor Reg(28) xor Reg(29) xor Reg(30) xor Reg(31) xor Data_insert(5);
   Reg(6) <= Reg(25) xor Reg(26) xor Reg(28) xor Reg(29) xor Reg(30) xor Reg(31) xor Data_insert(6);
   Reg(7) <= Reg(24) xor Reg(26) xor Reg(27) xor Reg(29) xor Reg(31) xor Data_insert(7);
   Reg(8) <= Reg(0) xor Reg(24) xor Reg(25) xor Reg(27) xor Reg(28);
   Reg(9) <= Reg(1) xor Reg(25) xor Reg(26) xor Reg(28) xor Reg(29);
   Reg(10) <= Reg(2) xor Reg(24) xor Reg(26) xor Reg(27) xor Reg(29);
   Reg(11) <= Reg(3) xor Reg(24) xor Reg(25) xor Reg(27) xor Reg(28);
   Reg(12) <= Reg(4) xor Reg(24) xor Reg(25) xor Reg(26) xor Reg(28) xor Reg(29) xor Reg(30);
   Reg(13) <= Reg(5) xor Reg(25) xor Reg(26) xor Reg(27) xor Reg(29) xor Reg(30) xor Reg(31);
   Reg(14) <= Reg(6) xor Reg(26) xor Reg(27) xor Reg(28) xor Reg(30) xor Reg(31);
   Reg(15) <= Reg(7) xor Reg(27) xor Reg(28) xor Reg(29) xor Reg(31);
   Reg(16) <= Reg(8) xor Reg(24) xor Reg(28) xor Reg(29);
   Reg(17) <= Reg(9) xor Reg(25) xor Reg(29) xor Reg(30);
   Reg(18) <= Reg(10) xor Reg(26) xor Reg(30) xor Reg(31);
   Reg(19) <= Reg(11) xor Reg(27) xor Reg(31);
   Reg(20) <= Reg(12) xor Reg(28);
   Reg(21) <= Reg(13) xor Reg(29);
   Reg(22) <= Reg(14) xor Reg(24);
   Reg(23) <= Reg(15) xor Reg(24) xor Reg(25) xor Reg(30);
   Reg(24) <= Reg(16) xor Reg(25) xor Reg(26) xor Reg(31);
   Reg(25) <= Reg(17) xor Reg(26) xor Reg(27);
   Reg(26) <= Reg(18) xor Reg(24) xor Reg(27) xor Reg(28) xor Reg(30);
   Reg(27) <= Reg(19) xor Reg(25) xor Reg(28) xor Reg(29) xor Reg(31);
   Reg(28) <= Reg(20) xor Reg(26) xor Reg(29) xor Reg(30);
   Reg(29) <= Reg(21) xor Reg(27) xor Reg(30) xor Reg(31);
   Reg(30) <= Reg(22) xor Reg(28) xor Reg(31);
   Reg(31) <= Reg(23) xor Reg(29);

   if (Checksum_Start = '1' and Counter = 3) then
            if (Reg = X"00000000") then
```

```
                    fcs_error <= '0';
                else
                    fcs_error <= '1';
                end if;
            Checksum_Start <= '0';
        end if;
    end if;
  end process;
end behavioral;
```

Listing 3: Parallel FCS Check

6 Synthesis and Place & route

A synthesis and place & route were also performed, which determined that the worst case operating frequency was 428.08 MHz at 1200mV and 85°C.

7 Testbench Parallel implmentation

The testbench used for verification is shown below and can also be accessed via github.

```
library ieee;
use ieee.std_logic_1164.all;

entity fcs_check_parallel_tb is
end fcs_check_parallel_tb;

architecture behavior of fcs_check_parallel_tb is

  -- componant used in testbench
  component fcs_check_parallel
    port(clk            : in  std_logic;
         reset          : in  std_logic;
         start_of_frame : in  std_logic;
         end_of_frame   : in  std_logic;
         data_in  : in  std_logic_vector(7 downto 0);
         fcs_error      : out std_logic);
  end component;
  -- Signals
  signal Test_clk            : std_logic := '0';
  signal Test_reset          : std_logic := '0';
  signal Test_start_of_frame : std_logic := '0';
  signal Test_end_of_frame   : std_logic := '0';
  signal Test_data_in        : std_logic_vector(7 downto 0) := x"00";
  signal Test_fcs_error      : std_logic;
```

```

  -- Clock Speed
  constant clk_period : time := 1 ns;

  --  Ethernet packet (FCS checksum E6 C5 3D B2)
  constant Ethernet_packet : std_logic_vector(511 downto 0) :=
    x"00_10_A4_7B_EA_80_00_12_34_56_78_90_08_00_45" &
    x"00_00_2E_B3_FE_00_00_80_11_05_40_C0_A8_00_2C" &
    x"C0_A8_00_04_04_00_04_00_00_1A_2D_E8_00_01_02" &
    x"03_04_05_06_07_08_09_0A_0B_0C_0D_0E_0F_10_11" &
    x"E6_C5_3D_B2";
begin
    Comp1 : fcs_check_parallel port map (
    clk            => Test_clk,
    reset          => Test_reset,
    start_of_frame => Test_start_of_frame,
    end_of_frame   => Test_end_of_frame,
    data_in        => Test_data_in,
    fcs_error      => Test_fcs_error
    );

  -- Clock generation
  Clk_Generator : process
  begin
    Test_clk <= '0';
    wait for clk_period / 2;
    Test_clk <= '1';
    wait for clk_period / 2;
  end process;

  -- Data_Simulation
  Data_Sim : process
  begin
    -- Start by reseting everything
    Test_reset <= '1';
    wait for clk_period;
    Test_reset <= '0';
    -- Begin sending the data
    for i in 63 downto 0 loop
      if i = 63 then
        Test_start_of_frame <= '1';
      else
        Test_start_of_frame <= '0';
      end if;
      Test_data_in <= Ethernet_packet(i*8+7 downto i*8);
      -- goes high when 32 bits remain(FCS)
```

```
      if i = 3 then
        Test_end_of_frame <= '1';
      else
        Test_end_of_frame <= '0';
      end if;
      wait for clk_period;
    end loop;
    Test_data_in <= x"00";
    wait;
  end process;
end;
```

Listing 4: Testbench of Parallel FCS Check

References

8 Appendix

8.1 Matlab for generation of CRC value

```

% polynomial
G = [1,0,0,0,0,0,1,0,0,1,1,0,0,0,0,0,1,0,0,0,1,1,1,0,1,1,0,1,1,0,1,1,1];

% degree
degree = 32;

%Data
Data = [0x00, 0x10, 0xA4, 0x7B, 0xEA, 0x80, 0x00, 0x12, ...
        0x34, 0x56, 0x78, 0x90, 0x08, 0x00, 0x45, 0x00, ...
        0x00, 0x2E, 0xB3, 0xFE, 0x00, 0x00, 0x80, 0x11, ...
        0x05, 0x40, 0xC0, 0xA8, 0x00, 0x2C, 0xC0, 0xA8, ...
        0x00, 0x04, 0x04, 0x00, 0x04, 0x00, 0x00, 0x1A, ...
        0x2D, 0xE8, 0x00, 0x01, 0x02, 0x03, 0x04, 0x05, ...
        0x06, 0x07, 0x08, 0x09, 0x0A, 0x0B, 0x0C, 0x0D, ...
        0x0E, 0x0F, 0x10, 0x11];

% convert from hex to binary vector
binStr = reshape(dec2bin(Data, 8).', 1, []); %Convert Hex to Binary string
M = double(binStr) - double('0'); % convert from ascii to numbers a binary vector

% inverts the first 32 bits
M(1:degree) = 1 - M(1:degree);

% Adding 32 zeros for the division
M = [M, zeros(1, degree)];

% Xor when the leading bit is '1' throgh out the data (exept the last 32 bits)
for i = 1 : (length(M) - degree)
    if M(i) == 1
        M(i:i+degree) = xor(M(i:i+degree), G);
    end
end

% Find the last 32 bits and invert them
R = M(end-degree+1 : end);
R_final = 1 - R; % Final bitwise complement
```

```
%Display
% Select the last 32 bits and placeing them in a 8x4 matrix
bit_matrix = reshape(R_final(1:32), 8, 4)';

powers_of_two = 2.^(7:-1:0)';% Creating a vector to convert to decimal
decimal_values = bit_matrix * powers_of_two; % Decimal value

hex_bytes = dec2hex(decimal_values, 2); % Convert from decimal to hex

% write to terminal
fprintf('Calculated Hex: 0x%s 0x%s 0x%s 0x%s\n', ...
hex_bytes(1,:), hex_bytes(2,:), hex_bytes(3,:), hex_bytes(4,:));
```

Listing 5: Matlab for generation of CRC value

8.2 Matlab for generating the parallel setup

```
% Parallel CRC implementation
G = [1,0,0,0,0,0,1,0,0,1,1,0,0,0,0,0,1,0,0,0,1,1,1,0,1,1,0,1,1,0,1,1,1];
Degree = 32;

% UpperLeft Construction
v = zeros(31, 1);
I = eye(31);
upper_section_left = [v, I];
UpperLeft = [upper_section_left; fliplr(G(2:end))];

% Remaining Matrix Parts
UpperRight  = zeros(32, 8);
A = zeros(1, 8); A(8) = 1;
B = zeros(7, 1); I2 = eye(7);
Bottom_right_merge = [I2, B];
BottomRight = [A; Bottom_right_merge];
BottomLeft = zeros(8, 32);
BottomLeft(1,1) = 1;

% Combine and calculate M^8
Combining = [UpperLeft, UpperRight; BottomLeft, BottomRight];
M2 = mod(Combining * Combining, 2);
M4 = mod(M2 * M2, 2);
Combining8 = mod(M4 * M4, 2);
```

```
% Generate VHDL code lines
for x = 1:size(Combining8, 2)
    if x > 32, break; end
    terms = {};
    for y = 1:size(Combining8, 1)
        if Combining8(y, x) == 1
            if y > 32
                % Map y=33..40 to data_in(7..0)
                terms{end+1} = sprintf('data_in(%d)', 40 - y);
            else
                terms{end+1} = sprintf('Reg(%d)', y - 1);
            end
        end
    end
    rhs = strjoin(terms, ' xor ');
    if isempty(terms), rhs = '0'; end
    fprintf('Reg(%d) <= %s;\n', x - 1, rhs);
end
```

Listing 6: Matlab for generating the parallel setup